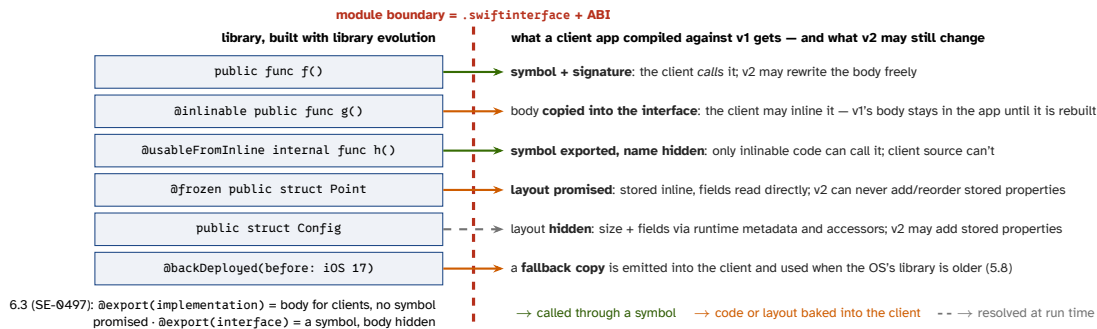


In one line: An **attribute** (`@name(args)`) changes how the compiler treats a declaration or type — what it exports, how it dispatches, what it checks — without changing its code; a **directive** (`#if`, `#warning`, `#fileID`...) acts at compile time on the source itself. The ones that matter most are about **boundaries**: module/ABI (`@inlinable`, `@frozen`), language (`@objc`), concurrency (`@preconcurrency`) and build configuration (`#if`).



Library evolution & optimisation

<code>@inlinable</code>	body is part of the public interface; may use only public or <code>@usableFromInline</code> declarations
<code>@usableFromInline</code>	on internal/package: ABI-visible, source-invisible
<code>@frozen</code>	public struct/enum: layout (or cases) fixed forever; ignored without library evolution
<code>@backDeployed(before:)</code>	ships a copy of a new API into clients on older OSes (Apple SDK-style libraries)
<code>@inline(usually) / (never)</code>	inlining hint / forbid (<i>underscored, unofficial</i>); <code>@inline(always)</code> (6.3, SE-0496) errors if a direct call can't be inlined, implies <code>@inlinable</code> on public API

API surface

<code>@discardableResult</code>	callers may ignore the return value without a warning
<code>@available(*, deprecated, renamed: "load(id:)"</code>	warning + Xcode fix-it; message:, unavailable (error), introduced:/obsoleted:, noasync
<code>@available(iOS 17, *)</code>	callers must be in an <code>if #available</code> block or be gated themselves
<code>@main</code>	the type's static <code>func main()</code> (may be <code>async throws</code>) is the entry point; <code>UIApplicationMain</code> is an error in Swift 6 mode (SE-0383)
<code>@testable import M</code>	tests see internal; needs <code>ENABLE_TESTABILITY</code> (Debug)
<code>@_spi(Beta) public package</code>	visible only to <code>@_spi(Beta) import</code> (<i>underscored, unofficial</i>)
<code>: @retroactive P</code>	access for modules of one package (5.9, SE-0366)
	you own neither type nor protocol (6.0, SE-0364)

Objective-C interop

<code>@objc / @objc(name)</code>	exposed to the ObjC runtime (selectors, <code>#selector</code> , KVC)
<code>@objcMembers</code>	<code>@objc</code> on every compatible member — incl. extensions and subclasses
<code>@nonobjc</code>	opt one member out
<code>@objc dynamic</code>	message dispatch via <code>objc_msgSend</code> : KVO, swizzling; dynamic alone is not KVO-observable
<code>@convention(c) / (block)</code>	function type with a C / ObjC-block ABI

Concurrency & closures (details: concurrency sheets)

<code>@preconcurrency import M</code>	downgrade Sendable diagnostics from an unmigrated module (5.6)
<code>: @preconcurrency P</code>	conformance whose isolation is checked at run time (6.0, SE-0423)
<code>@unchecked Sendable</code>	"trust me": no checking of stored properties — you own the locking
<code>@TaskLocal static var id</code>	value bound per task tree: <code>\$id.withValue("r1") { _ }</code> ; child tasks inherit, <code>Task.detached</code> doesn't
<code>@escaping @Sendable</code>	closure type: may outlive the call · may run concurrently · runs on the main actor
<code>@MainActor () -> Void</code>	opt a member out of its actor · opt out of checking
<code>nonisolated · (unsafe)</code>	parameter ownership; consume <code>x</code> ends a lifetime, copy <code>x</code> copies explicitly (5.9)
<code>consuming / borrowing</code>	

Type roles, Interface Builder, Core Data

<code>@propertyWrapper · @resultBuilder</code>	the type becomes a wrapper / a DSL builder (own sheets)
<code>@dynamicMemberLookup · @dynamicCallable</code>	<code>x.name / x(1, 2)</code> rewritten to subscripts / calls
<code>@IBOutlet · @IBAction</code>	storyboard connections; imply <code>@objc</code>
<code>@NSManaged var</code>	Core Data supplies the accessors at run time
<code>@NSCopying var</code>	the setter stores <code>newValue.copy()</code>
<code>@warn_unqualified_access</code>	warn unless called as <code>self.min()</code> — avoids the global <code>min</code>

Related: swift-keypaths-access-control (access levels) · swift-enums-pattern-matching (`@frozen` enums) · macros-and-result-builders · swift6-strict-concurrency · objc-interop · modularization-spm · swift-syntax-closures-calls

Compiler directives

```
#if os(iOS) || os(visionOS) // os arch canImport targetEnvironment
#if canImport(UiKit) && !targetEnvironment(macCatalyst)
#if targetEnvironment(simulator) // simulator | macCatalyst
#if compiler(>=6.0) // the COMPILER version (any language mode)
#if swift(>=6.0) // the LANGUAGE MODE (-swift-version 6)
#if hasFeature(ExistentialAny) // 5.8 SE-0362; hasAttribute(retroactive)
#if DEBUG // your -D flag (SWIFT_ACTIVE_COMPILATION_CONDITIONS)
Text("Hi")
#if os(iOS)
.padding() // 5.5 SE-0308: #if around postfix members
#endif
#warning("TODO: remove") // #error("...") stops the build
#sourceLocation(file: "gen.swift", line: 1) // generated code maps back
#fileID // "App/Feed.swift"; #file = #fileID in Swift 6 mode
#filePath // full build-machine path - baked into the binary!
#function // "load(id:)" #line #column #dsohandle: image handle
func run(isolation: isolated (any Actor)? = #isolation) // caller's actor
macro stringify<T>(_ v: T) -> (T, String) =
#externalMacro(module: "Macros", type: "StringifyMacro")
```

Traps

- ! `swift(>=6)` is the language *mode*; a Swift 6 compiler in 5 mode fails it but passes `compiler(>=6)`.
- ! Inactive `#if` branches must still *parse* — except under `swift()/compiler()` checks.
- ! `@inlinable` fixes reach clients only when they rebuild; `@frozen` is forever.
- ! `@unchecked Sendable` silences the checker; it proves nothing.
- ! `@Observable`, `@Model` look like attributes but are **macros** (they expand into code); `@MainActor` is a global-actor attribute.

Remember

"Attributes move boundaries: module (`@inlinable`, `@frozen`), language (`@objc`), isolation (`@preconcurrency`); `#if` moves code out of the build."